时光像素 — 完整代码级决策流程

> 以用户输入"装饰器"（八股事件）为例，追踪到 HTML 渲染。

0. main.py — 接收 WebSocket 消息

```
前端: ws.send('{"event":"装饰器"}')
  ↓
main.py:62  data = await websocket.receive_json()
main.py:63  user_input = data.get("event", "").strip()  → "装饰器"
main.py:65  if not user_input: return send_failed(...)  → 不为空，继续
main.py:70  send_progress("system","running", f"收到事件：「{user_input}」")
main.py:80  state = initial_state("装饰器")
```

`initial_state("装饰器")` state.py:56 → 返回 dict：

```
{
  "user_input": "装饰器",
  "puzzle_type": "",           # 待填
  "puzzle_design": {},
  "search_results": [],
  "material_score": 0.0,
  "material_sufficient": False,
  "game_script": "",
  "script_data": {},           # 刚修的 bug
  "script_keywords": [],
  "game_code": "",
  "review_passed": False,
  "review_feedback": "",
  "review_details": {},
  "retry_count": 0,
  "styled_code": "",
  "visual_css": "",
  "directions": [],
  "selected_direction": {},
  "status": "running",         # ← 关键：初始是 running
  "error_message": "",
  "suggestions": [],
  "agent_logs": [],
}
```

1. CRAWLER — crawler.py:crawler_node(state)

```
workflow.py: crawler → planner → writer → artist_pre → coder → reviewer →
artist_post
第一个节点是 crawler
```

1.1 查 KB

```
# crawler.py:62
user_input = state["user_input"]  → "装饰器"
verified_event = get_event_by_keyword("装饰器")
```

`get_event_by_keyword("装饰器")` kb.py:31-55：

```
text_lower = "装饰器"
best_match = None, best_score = 0

# 遍历 27 计算机历史事件 + 8 八股事件 = 35 个
for event in all_events:  # 每个事件尝试匹配

    # 1) keywords 匹配
    for kw in event.get("keywords", []):
        if kw.lower() in text_lower:    # "装饰器" in "装饰器" → True
            score += 1                  # 每个匹配 +1

    # 2) aliases 匹配
    for alias in event.get("aliases", []):
        if alias.lower() in text_lower:
            score += 1

    # 3) 事件名子串匹配
    name = event.get("event", event.get("title", ""))  # "装饰器语法糖"
    for word in text_lower.split():   # ["装饰器"]
        if len(word) >= 2 and word in name.lower():
            score += 0.5    # "装饰器" 是 "装饰器语法糖" 的子串 → +0.5

    # 对于 装饰器语法糖 事件:
    # keyword "装饰器" 匹配 → +1
    # 子串 "装饰器" 在 title 中 → +0.5
    # total = 1.5 ≥ 1 → 命中!

if score >= 1:
    return best_match  → 返回 装饰器语法糖 事件
```

1.2 转 search_results

```
# crawler.py:64
verified_event = 装饰器语法糖事件
kb_sources = event_to_search_results(verified_event)
```

`event_to_search_results()` kb.py:79-101：

```
# 检测到 content.original 存在 → 走八股分支
content = event["content"]  # {original, translation, annotations}
return [{
    "title": "「装饰器语法糖」",
    "content": "装饰器是一个返回函数的高阶函数...\n\n原始代码：\ndef
my_decorator(func):...",
    "confidence": "high",
```

```
    "verified": True,
    "source": "verified_knowledge_base",
    "key_facts": ["@my_decorator 等价于 say_hello = my_decorator(say_hello)",
...],
    "atmosphere_tags": ["终端","代码","IDE"],
    "key_props": ["终端","光标","代码高亮"],
    "visual_anchor": "终端中 @ 符号高亮闪烁...",
    "category": "bagu",
    "puzzle_guide": {              # ← 完整谜题配置
        "type": "fill_blank",
        "blanks": [...],
        "hints": [...],
        "expected_output": {...},
        "scoring": {...}
    },
}]
```

1.3 跳过 web_search

```
# crawler.py:69
is_predefined = bool(verified_event and verified_event.get("puzzle_guide"))
# verified_event.puzzle_guide 存在 → is_predefined = True

if not is_predefined:
    # 八股事件不执行这里
    web_results = web_search(user_input, max_results=5)
else:
    web_results = []  # 跳过
```

1.4 判断素材是否足够

```
# crawler.py:78
total_chars = sum(len(s.get("content","")) for s in all_sources)
# all_sources 只有 KB 那一条，content 大约 200-400 字符

# crawler.py:81
if is_predefined:
    # 直接返回，不管 total_chars 够不够
    return {
        "search_results": all_sources,  # 1 条
        "material_score": 1.0,
        "material_sufficient": True,
        "agent_logs": [agent_log("crawler","verified",
            "predefined puzzle: fill_blank, skip web_search")],
    }
```

决策结果：crawler 返回，state 被更新为：

```
state["search_results"] = [...]     # 1 条八股资料
state["material_score"] = 1.0
state["material_sufficient"] = True # ← workflow 的 should_continue_after_crawler
读这个
```

1.5 Workflow 路由

```
# workflow.py:27
def should_continue_after_crawler(state):
    if state["material_sufficient"]:  → True
        return "planner"              # → 进入 planner
    return "end_failed"
```

---

2. PLANNER — planner.py:planner_node(state)

2.1 检测预定义题型

```
# planner.py:47-48
user_input = state["user_input"]     → "装饰器"
search_results = state.get("search_results", [])  → 1 条

# planner.py:51-62
for r in search_results:
    pg = r.get("puzzle_guide", {})
    if pg and pg.get("type"):        # "fill_blank" → True
        # 跳过 LLM!
        return {
            "puzzle_type": "fill_blank",
            "puzzle_design": {
                "mechanic": "Python 面试 - fill_blank",
                "rules": "; ".join(pg.get("annotations", r.get("key_facts",[])))
[:200],
                "win_condition": "所有空位填写正确",
            },
            "material_sufficient": True,
            "agent_logs": [agent_log("planner","predefined","type=fill_blank from
KB")],
        }
```

决策结果：不调 LLM，puzzle_type 直接从 KB 读取。省 1 次 LLM。

```
state["puzzle_type"] = "fill_blank"
state["puzzle_design"] = {...}
state["material_sufficient"] = True
```

2.2 Workflow 路由

```
# workflow.py:33-37
def should_continue_after_planner(state):
    if state["material_sufficient"]:  → True
        return "writer"               # → 进入 writer
    return "end_failed"
```

---

3. WRITER — writer.py:writer_node(state)

3.1 检测八股类型

```
# writer.py:68-72
user_input = state["user_input"]     → "装饰器"
puzzle_type = state["puzzle_type"]   → "fill_blank"
search_results = state.get("search_results", [])

is_bagu = puzzle_type in ("fill_blank","recite","match","debugger")
# "fill_blank" 在集合里 → is_bagu = True
```

3.2 拼史料

```
# writer.py:77-93
parts = []
for r in search_results[:3]:
    title = r.get('title', '')
    story = r.get('content', '')    # 八股事件：translation + original 代码
    facts = r.get('key_facts', [])  # 八股事件：annotations

    block = f"【{title}】\n"
    if story and len(story) > 50:   # 有代码+翻译 → True
        block += story
    elif facts:
        block += "; ".join(facts)

    # 追加 V4 字段
    if r.get("atmosphere_tags"):
        block += f"\n氛围标签：{'、'.join(r['atmosphere_tags'])}"
    if r.get("key_props"):
        block += f"\n关键道具：{'、'.join(r['key_props'])}"
    if r.get("visual_anchor"):
        block += f"\n视觉锚点：{r['visual_anchor']}"
    parts.append(block)

sources_text = "\n\n".join(parts)
```

3.3 组装 prompt + 调 LLM

```
# writer.py:107
prompt = f"""历史事件：装饰器
谜题类型：fill_blank
谜题机制：Python 面试 - fill_blank
规则：@my_decorator 等价于...

史料：
【「装饰器语法糖」】
装饰器是一个返回函数的高阶函数...
原始代码：
def my_decorator(func):...
氛围标签：终端、代码、IDE
关键道具：终端、光标、代码高亮
```

```
视觉锚点：终端中 @ 符号高亮闪烁...

请输出完整 GameScript JSON。puzzle.type 必须是 fill_blank。"""

# writer.py:121
system = BAGU_SYSTEM_PROMPT if is_bagu else SYSTEM_PROMPT
# is_bagu=True → 使用 BAGU_SYSTEM_PROMPT

raw = chat(prompt, system=BAGU_SYSTEM_PROMPT, temperature=0.5)
```

3.4 清理 + 解析

```
# writer.py:122-123
cleaned = _strip_markdown_fence(raw)
script = json.loads(cleaned)
# script = {
#   "event": "装饰器语法糖",
#   "year": 2,
#   "protagonist": "装饰器语法糖",
#   "antagonist": "@ 符号容易写错位置",
#   "atmosphere": "终端,代码,IDE",
#   "opening_hook": "...",
#   "puzzle": {"type":"fill_blank","surface":"...","hints":
[...],"max_attempts":3},
#   "history_facts": {"title":"...","story":"200-300字讲
解","key_point":"...","fun_fact":"..."},
#   "victory_line": "Process finished with exit code 0",
#   "defeat_line": "SyntaxError: decorator not found",
#   "visual": {"palette":[...],"mood":"终端IDE","decorations":[...]},
#   "content": {"original":"def
my_decorator(func):...","translation":"...","annotations":[...]}
# }
```

3.5 写入 State

```
# writer.py:124-128
return {
    "game_script": json.dumps(script, ensure_ascii=False),
    "script_data": script,          # ← 写入 State（刚修，之前会被 LangGraph 丢弃）
    "script_keywords": ["装饰器", "fill_blank"],
    "agent_logs": [agent_log("writer","script_written","topic=装饰器语法糖,
chars=NNN")],
}
```

LangGraph 合并到 State：

```
state["game_script"] = '{"event":"装饰器语法糖",...}'  # JSON 字符串
state["script_data"] = {...}                            # dict
state["script_keywords"] = ["装饰器","fill_blank"]
state["agent_logs"].append(...)
```

4. ARTIST_PRE — artist_pre.py:artist_pre_node(state)

4.1 读剧本

```
# artist_pre.py:104-107
script = state.get("script_data", {})   # ✅ 现在能读到了（刚修的 bug）
puzzle_type = script.get("puzzle", {}).get("type", "cipher")
# script["puzzle"]["type"] = "fill_blank" → puzzle_type = "fill_blank"
```

4.2 调 LLM 生成视觉方向

```
# artist_pre.py:108-116
prompt = f"""请为以下历史游戏生成 2 个视觉设计方向。

事件：装饰器语法糖
类型：fill_blank
氛围：终端,代码,IDE
情绪：
时代：
道具：终端, 光标, 代码高亮

严格按 system prompt 的 JSON 格式输出。"""

# artist_pre.py:119-120
response = chat(prompt, system=SYSTEM_PROMPT, temperature=0.5)
response = _strip_markdown_fence(response)
data = json.loads(response)
directions = data.get("directions", [])
```

4.3 验证方向

```
# artist_pre.py:125-128
valid, msg = validate_directions(directions, puzzle_type)
# 检查：必须 2 个方向，每个方向必须有
name/mood_tags/palette/ui/animation/reference_css/post
# palette 必须是 5 个色值

if not valid:
    raise ValueError(f"验证失败: {msg}")
    # LLM 生成的格式不对 → 跳到 except，用默认方向
```

4.4 关键词硬匹配选方向

```
# artist_pre.py:129
selected = select_direction(directions, script)

# select_direction 内部：
# artist_pre.py:62-71
mood = script.get("mood", "")          # ""
atmo = script.get("atmosphere", "")    # "终端,代码,IDE"
event = script.get("event", "")        # "装饰器语法糖"
```

```
scores = []
for d in directions:
    score = calculate_mood_score(d, mood, atmo)
    # expand_mood_tags(["终端","代码"]) → {"终端","代码"} (无同义词)
    # combined = ("终端,代码,IDE").lower()
    # 匹配数 = "终端" in combined + "代码" in combined = 2
    scores.append((score, d))

scores.sort(reverse=True)

if scores[0][0] > scores[1][0]:  # 有差距 → 选高分
    return scores[0][1]
else:
    # 平局 → hash 选
    hash_val = int(hashlib.md5("装饰器语法糖".encode()).hexdigest(), 16)
    return directions[hash_val % len(directions)]
```

4.5 如果 LLM 失败了

```
# artist_pre.py:137-143
except Exception as e:
    fallback = DEFAULT_DIRECTIONS.get(puzzle_type, DEFAULT_DIRECTIONS["cipher"])
    # DEFAULT_DIRECTIONS["fill_blank"] = [
    #   {"name":"VS Code 暗色","palette":["#0d1117","#58a6ff",...],"ui":"..."},
    #   {"name":"终端绿屏","palette":["#0d1117","#7ee787",...],"ui":"..."},
    # ]
    selected = select_direction(fallback, script)
```

4.6 写入 State

```
return {
    "directions": directions,
    "selected_direction": selected,
    # selected = {"name":"VS Code 暗色","palette":
[...],"ui":"...","animation":"...","reference_css":"...","post":{...}}
    "agent_logs": [...],
}
```

---

5. CODER — coder.py:coder_node(state)

5.1 检测八股类型

```
# coder.py:110-114
puzzle_type = state["puzzle_type"]     → "fill_blank"
script_data = state.get("script_data", {})
direction = state.get("selected_direction", {})
review_feedback = state.get("review_feedback", "")  → ""
search_results = state.get("search_results", [])

is_bagu = puzzle_type in BAGU_TEMPLATES  → True
```

5.2 提取 puzzle_guide

```
# coder.py:118-123
if is_bagu:
    puzzle_guide = {}
    for r in search_results:
        if r.get("puzzle_guide"):
            puzzle_guide = r["puzzle_guide"]
            break
    # puzzle_guide = {
    #   "type": "fill_blank",
    #   "blanks": [{"code":"def my_decorator(___)","answer":"func",...}, ...],
    #   "hints": [{"level":1,"text":"装饰器接收一个函数作为参数"}, ...],
    #   "expected_output": {"error":null,"warning":{...},"success":"输出: ..."},
    #   "scoring": {"base_score":150,...}
    # }

    bagu_data_block = f"""
=== Python 面试题数据 ===
{json.dumps(puzzle_guide)}

=== 原始代码 ===
def my_decorator(func):
    def wrapper(*args, **kwargs):
        ...

=== 知识点 ===
["@my_decorator 等价于 say_hello = my_decorator(say_hello)", ...]
"""
```

5.3 组装 prompt

```
# coder.py:124-158（计算机历史部分先跳过）
# 八股走这里：

direction_block = f"""
=== 选定的视觉方向 ===
名称：VS Code 暗色
色板：#0d1117, #58a6ff, #7ee787, #e6edf3, #30363d
UI风格：VS Code风格编辑器，行号灰色，关键字蓝色语法高亮，空位闪烁下划线
动画节奏：代码填对时逐行变绿，终端输出模拟结果
参考CSS：
.rune{border:1px solid #58a6ff;color:#58a6ff;font-size:13px}.panel{...}
"""

prompt = f"""请按契约生成「fill_blank」类型的时间解谜游戏。
...（叙事信息、谜题参数、提示层级、历史真相、台词）...
"""

final_prompt = prompt + direction_block + bagu_data_block
final_system = BAGU_TEMPLATES["fill_blank"]  # FILL_BLANK_TEMPLATE
temp = 0.1  # 八股用更低温度
```

`FILL_BLANK_TEMPLATE` 关键内容：

- 带行号的代码编辑器样式
- 语法高亮（关键字蓝、字符串绿、注释灰）
- 空位用闪烁 ___ 表示
- 点击空位 → 内联替换为 `<input>`
- 填对 → 绿色 + terminal 输出 success
- 填错 → 红色 + 模拟 Python 报错
- 数据从 `window.__PUZZLE_DATA__` 读取

5.4 生成代码

```
# coder.py:220-222
code = chat(final_prompt, system=final_system, temperature=0.1)
code = _strip_markdown_fence(code)
if not code.lower().startswith("<!doctype"):
    code = f"<!DOCTYPE html>\n{code}"
```

5.5 写入 State

```
return {
    "game_code": code,  # 完整的 HTML 字符串
    "agent_logs": [agent_log("coder","code_generated",f"{len(code)} chars")],
}
```

---

6. REVIEWER — reviewer.py:reviewer_node(state)

6.1 Phase 1：正则机械检查

```
# reviewer.py:86-92
game_code = state["game_code"]
retry_count = state.get("retry_count", 0) + 1  → 1

p1 = phase1_contract_check(game_code)
```

`phase1_contract_check()` 执行的检查：

CRITICAL_RULES（缺 1 个 → 致命）：

```
1. r'<!DOCTYPE\s+html'                                  → ✅ 匹配
2. r'<script[^>]*>'                                     → ✅ 匹配
3. r'id=["\']screen-game["\']'                          → ⚠️ 八股模板可能没有
4. r'(const\s+gameState|let\s+gameState|...) '      → ⚠️ 八股模板可能用
window.__PUZZLE_DATA__
```

WARNING_RULES（缺了继续）：

```
5. r'(id=["\']screen-result["\']|胜利|失败|...) '  → 检查
6. r'(HISTORY_FACTS|历史真相)'                        → ⚠️ 八股模板可能叫"知识点"
```

CSS_QUALITY_CHECKS（CSS bug 检测）：

```
7. r'cursor\s*:\s*pointer'                          → 必须有
8. r'pointer-events\s*:\s*none'                     → 必须没有（或只在 .screen 上）
```

决策:

```
if critical_missing:
    return {"level":"CRITICAL","pass":False}  → 跳过 Phase2,直接打回 coder

if warning_missing or css_warnings:
    return {"level":"WARNING","pass":False}   → 继续 Phase2 但带警告
```

6.2 Phase 2：LLM 质量审查（仅非 CRITICAL 时）

```
# reviewer.py:144-170（如果 Phase1 是 PASS 或 WARNING）

prompt = f"""审查这个 {len(game_code)} 字符的 HTML 游戏。
【游戏剧本】{game_script[:500]}
【原始史料】{sources}
【完整游戏代码】{game_code}
当前重试: {retry_count}/{MAX_REVIEW_RETRIES}..."""

response = chat_json(prompt, system=SYSTEM_PROMPT)
result = json.loads(response)

passed = result.get("passed", False)  → True（假设通过）
```

6.3 写入 State

```
if passed:
    return {
        "review_passed": True,
        "review_feedback": result.get("feedback", ""),
        "review_details": {...},
        "retry_count": retry_count,
        "agent_logs": [agent_log("reviewer","pass","...")],
    }
else:
    # retry_count < 3: 打回 coder 重写
    # retry_count = 3: status="failed", 终止
```

6.4 Workflow 路由

```
# workflow.py:42-46
def should_continue_after_reviewer(state):
    if state["review_passed"]:  → True
        return "artist_post"    # → 进入 artist_post
    if state["retry_count"] < MAX_REVIEW_RETRIES:
        return "coder"          # 打回重写
    return "end_failed"
```

---

7. ARTIST_POST — artist_post.py:artist_post_node(state)

7.1 正则注入（强制）

```
# artist_post.py:79-89
game_code = state["game_code"]
direction = state.get("selected_direction", {})

styled = game_code

# 1. 注入 CSS 变量
styled = inject_palette_vars(styled, direction)
# :root{--bg:#0d1117;--primary:#58a6ff;--success:#7ee787;--text:#e6edf3;--
muted:#30363d}

# 2. 补 .screen transition
styled = inject_screen_transition(styled)
# .screen{opacity:0;transform:scale(0.98);transition:opacity 0.5s,transform
0.5s;...}
# .screen.active{opacity:1;transform:scale(1);pointer-events:auto}

# 3. 注入字体
styled = inject_fonts(styled)
# <link href="https://fonts.googleapis.com/css2?family=Press+Start+2P">

# 4. 注入 CRT/氛围
styled = inject_atmosphere(styled, direction)
# 按 direction.post.crt / particles / atmosphere 注入
```

7.2 可选 LLM 微调

```
# artist_post.py:91-103
try:
    style_match = re.search(r'<style>(.*?)</style>', styled, re.DOTALL)
    if style_match:
        existing = style_match.group(1)
        supplement = llm_generate_supplement(existing, direction)
        # LLM 只看 CSS 文本，生成补充 CSS
        if supplement:
            styled = styled.replace("</style>",
                f'\n/* === artist_post supplement ===
*/\n{supplement}\n</style>')
except Exception:
    pass  # LLM 挂了不影响，正则注入结果已可用
```

7.3 写入 State

```
return {
    "styled_code": styled,
    "status": "success",        # ← 前端判断 game_ready 的关键
    "agent_logs": [agent_log("artist_post","styled",f"{len(styled)} chars")],
}
```

---

8. main.py — 推送结果

8.1 累积最终状态

```
# main.py:88-89
final_output = {}

# astream_events 循环中，每个 on_chain_end 事件：
# main.py:159-160
if isinstance(output, dict):
    final_output.update(output)

# 最后一个节点 artist_post 的 output 被 update 进来：
# final_output["styled_code"] = "..."
# final_output["status"] = "success"
```

8.2 推送

```
# main.py:164-177
cost = get_cost_summary()
logger.info(f"生成完成，本次花费: ¥{cost['estimated_cost_rmb']} ({cost['calls']}次
LLM调用)")
logger.info(f"final_output keys: {list(final_output.keys())} status=
{final_output.get('status')}")

if final_output.get("status") == "success":   # "success" → True
    await ws_manager.send_game_ready(session_id, final_output.get("styled_code",
""))
    # 前端收到 {"type":"game_ready","game_code":"<!DOCTYPE html>..."}
else:
    await ws_manager.send_failed(
        session_id,
        final_output.get("error_message", "生成失败"),
        final_output.get("suggestions", []),
    )
```

8.3 前端

```
// useWebSocket.js:61-64
case 'game_ready':
    setGameCode(data.game_code)       // 设置 gameCode
    setIsGenerating(false)            // 解除生成状态
    generatingRef.current = false
```

```
    break

// App.jsx:91-98
<GamePanel
    visible={!!gameCode}              // gameCode 非空 → 显示游戏面板
    gameCode={gameCode}               // iframe srcDoc 的内容
    isGenerating={isGenerating}
    ...
/>
```

决策点总表（23 个）

#	文件:行	决策	条件	A 分支	B 分支
1	main:65	输入为空?	`not user_input`	send_failed	继续
2	crawler:63	KB 命中?	`score >= 1`	继续	DeepSeek 兜底
3	kb:79	事件类型?	有 `content.original`	八股分支	计算机历史分支
4	crawler:69	预定义题型?	`puzzle_guide` 存在	跳过 web_search	web_search
5	crawler:81	素材够?	`is_predefined`	直接返回	检查 chars
6	crawler:80	普通事件够?	`total_chars >= 300`	直接返回	DeepSeek
7	workflow:27	crawler 后?	`material_sufficient`	进 planner	end_failed
8	planner:53	预定义题型?	`puzzle_guide.type` 存在	跳过 LLM	调 LLM
9	workflow:33	planner 后?	`material_sufficient`	进 writer	end_failed
10	writer:72	八股类型?	puzzle_type 在 bagu 集合	BAGU_PROMPT	HISTORY_PROMPT
11	writer:122	JSON 解析?	`json.loads` 成功	script_data	fallback 模板
12	artist_pre:125	方向验证?	`validate_directions`	继续	默认方向
13	artist_pre:68	选哪个方向?	mood 匹配数 A vs B	选高分	hash 平局
14	coder:118	八股类型?	puzzle_type 在 BAGU_TEMPLATES	bagu 模板	历史模板
15	coder:222	温度?	is_bagu	temp=0.1	temp=0.3
16	reviewer:99	Phase1 致命?	critical_missing 非空	直接打回	继续检查
17	reviewer:99	Phase1 警告?	warning_missing 非空	记警告+继续	PASS
18	reviewer:161	Phase2 通过?	passed	进 artist_post	检查重试
19	reviewer:122	重试上限?	`retry_count >= 3`	失败终止	打回 coder

#	文件:行	决策	条件	A 分支	B 分支
20	workflow:42	reviewer 后?	`review_passed`	进 artist_post	打回/终止
21	artist_post:91	LLM 微调?	try 成功	追加 CSS	静默跳过
22	main:167	推送什么?	`status == "success"`	game_ready	generation_failed
23	App.jsx:91	游戏显示?	`!!gameCode`	显示面板	隐藏